

YOLO

You Only Look Once

(“Você **Vê** [•imagens, •frames de vídeos] Apenas Uma vez”)

Se o YOLO **vê**:

- ✓ É porque ele é uma ferramenta...
- ✓ De **visão** computacional.

Basicamente:

- ✓ A imagem inteira...
- ✓ Passa uma única vez.
- ✓ Pela **rede neural**...

O que é uma **rede neural**???

É um sistema de IA
inspirado no cérebro humano.

Nosso cérebro:

- ✓ Recebe informações...
- ✓ Processa essas informações...
- ✓ E gera uma resposta.

De forma similar, uma rede neural:

- ✓ Recebe informações (linhas, cores...)...
- ✓ Processa essas informações (formas, padrões...)...
- ✓ E gera uma resposta (Ex.: “Isso é uma planta.”).

Neste projeto, o objetivo foi:

- ✓ Desenvolver um **modelo** de IA...
- ✓ Que fosse capaz de identificar...
- ✓ Diferentes espécies de **PANCs**.

Modelo → IA já treinada:

- ✓ Que reconhece padrões em imagens...
- ✓ E que pode ser ajustada para aprender novas tarefas.

O que são **PANCs**???

PANCs

Plantas Alimentícias Não Convencionais

PANCs são plantas:

- ✓ Que, geralmente, não são consumidas no dia a dia...
- ✓ Mas que podem ser usadas:
 - Como alimento...
 - Como medicamento...
- ✓ E que podem (também) ser encontradas em áreas urbanas.



Azedinha (*Rumex acetosa*) → Sabor ácido e refrescante, semelhante ao do limão (é rica em vitamina C) (boa para saladas, molhos, sopas)

Neste projeto, foram escolhidas 5 (cinco) PANCs:



Ora-pro-nóbis



Peixinho



Lírio (gingibre) do brejo



Mastruz



Coração de bananeira

- Foram escolhidas 500 imagens no total, • sendo 100 imagens para cada PANC – • e destas, foram separadas:
 - ✓ 80 imagens para treinamento...
 - ✓ 20 imagens para teste...
 - ✓ Para treinar um modelo de **classificação** de imagens.

Tarefa	Responde à pergunta	Exemplo
Classificação (discreta: resulta numa categoria)	O que há na imagem?	"É Ora-pro-nóbis."
Detecção	O que há e onde está?	"Há 2 Ora-pro-nóbis e 1 Peixinho nestas posições."
Segmentação	Qual é o formato exato do objeto?	Separa cada folha do fundo pixel a pixel
Pose Estimation	Onde estão os pontos importantes?	Ponta da folha, caule, flor.
Tracking	É o mesmo objeto ao longo do vídeo?	Acompanha a mesma planta em vários frames
Contagem	Quantos objetos existem?	23 Ora-pro-nóbis, 17 Peixinhos
OCR (Optical Character Recognition)	Que texto aparece na imagem?	Lê " <i>Rumex acetosa</i> " numa etiqueta
Similaridade Visual	Qual imagem é mais parecida?	Busca espécies semelhantes à foto enviada
Detecção de Anomalias	Há algo fora do normal?	Detecta doenças, fungos ou pragas
Regressão (prevê um valor numérico contínuo)	Qual é o valor numérico?	"É Ora-pro-nóbis." "Altura: 42 cm."

```

1 ##### PANCs --> Plantas Alimentícias Não Convencionais
2
3 # pip install ultralytics --> Biblioteca usada para treinamento de modelos de visão computacional
4
5 # ora-pro-nobis --> Pereskia aculeata
6 # peixinho --> Stachys byzantina
7 # gengibre-do-brejo --> Hedychium coronarium
8 # mastruz --> Dysphania ambrosioides
9 # coracao-de-bananeira --> Musa acuminata
10
11 from ultralytics import YOLO
12 # YOLO (classe):
13 #   * You Only Look Once
14 #   * Aqui, a rede neural analisa a imagem inteira em uma única passada
15 import os # --> Módulo padrão do Python que dá acesso a pastas, arquivos etc.
16
17 BASE_DIR = os.path.dirname(__file__)
18 # BASE_DIR:
19 #   * Pega a pasta onde o arquivo .py está localizado
20 #   * Está em maiúscula porque é uma constante (é um valor que não deve mudar)
21 # __file__ --> Variável especial do Python que representa o caminho do arquivo atual
22 # os.path.dirname(...) --> Função que remove o nome do arquivo e deixa apenas o nome do diretório
23
24 model = YOLO("yolo11n-cls.pt") # --> Objeto (instância da classe YOLO)
25 # YOLO( ) --> Cria um modelo YOLO
26 # "yolo11n-cls.pt" --> carrega um modelo pré-treinado.
27 # 11 --> versão do YOLO
28 # n --> Diferentes tamanhos de modelos:
29 #   * n, s, m, l, x --> nano, small, medium, large, extra-large
30 #   * nano (celulares), small (tempo real leve), medium (uso geral), large (GPU poderosa), extra-large (máxima precisão)
31 # cls --> Classification (para classificação de imagens)
32 # .pt --> Arquivo de pesos do PyTorch (biblioteca usada para criar e treinar *redes neurais e *modelos de IA)
33
34 results = model.train( # --> Inicia ou prepara o modelo para o treinamento
35     data=BASE_DIR, # --> Especifica o local dos dados de treinamento (se fosse apenas ".", seria o local do diretório atual)
36     epochs=10, # --> Define o número de épocas (ciclos completos de treinamento) para o modelo
37     # (20 é um número comum, mas 10 é usado aqui para um teste rápido)
38     # No hub.ultralytics.com, pode-se treinar por tempo (e não por épocas) --> 1 hora, por exemplo
39     # Vide --> https://www.youtube.com/watch?v=5B00Il\_YYAg
40     imgsz=128, # --> Define o tamanho das imagens usadas para treinamento (128x128 [16.384 pixels])
41     # (224x224 [50.176 pixels] é um tamanho comum, mas 128 é usado aqui para um teste rápido)
42     project=os.path.join(BASE_DIR, "pancs"), # Define a pasta "pancs" dentro do diretório base do projeto
43     name="train-10-epocas",
44     exist_ok=True, # Se a pasta do treino já existir (no caso, train-10-epocas), o código não dará erro
45     plots=True # Gera gráficos automáticos do treinamento
46 )
47
48 save_dir = results.save_dir # Guarda o caminho da pasta onde o treino foi salvo ("Onde estão os resultados do treinamento???)")
49
50 os.startfile(os.path.join(save_dir, "results.png"))
51 # os.startfile --> Abre automaticamente um arquivo como se alguém tivesse clicado nele
52 # os.path.join(...) --> Função que junta corretamente as partes de um caminho de arquivo
53 os.startfile(os.path.join(save_dir, "confusion_matrix.png"))
54 os.startfile(os.path.join(save_dir, "confusion_matrix_normalized.png"))

```

Epoch 1/10	GPU_mem 0G	loss 1.653	Instances 16	Size 128: 100%	25/25 9.2it/s 2.7s
	classes	top1_acc	top5_acc:	100%	4/4 17.4it/s 0.2s
	all	0.57	1		
Epoch 2/10	GPU_mem 0G	loss 0.971	Instances 16	Size 128: 100%	25/25 10.1it/s 2.5s
	classes	top1_acc	top5_acc:	100%	4/4 19.7it/s 0.2s
	all	0.76	1		
Epoch 3/10	GPU_mem 0G	loss 0.6264	Instances 16	Size 128: 100%	25/25 10.2it/s 2.5s
	classes	top1_acc	top5_acc:	100%	4/4 18.7it/s 0.2s
	all	0.79	1		
Epoch 4/10	GPU_mem 0G	loss 0.4568	Instances 16	Size 128: 100%	25/25 9.2it/s 2.7s
	classes	top1_acc	top5_acc:	100%	4/4 15.8it/s 0.3s
	all	0.85	1		
Epoch 5/10	GPU_mem 0G	loss 0.33	Instances 16	Size 128: 100%	25/25 10.3it/s 2.4s
	classes	top1_acc	top5_acc:	100%	4/4 18.5it/s 0.2s
	all	0.86	1		
Epoch 6/10	GPU_mem 0G	loss 0.2777	Instances 16	Size 128: 100%	25/25 10.6it/s 2.4s
	classes	top1_acc	top5_acc:	100%	4/4 17.1it/s 0.2s
	all	0.82	1		
Epoch 7/10	GPU_mem 0G	loss 0.2662	Instances 16	Size 128: 100%	25/25 9.8it/s 2.6s
	classes	top1_acc	top5_acc:	100%	4/4 15.6it/s 0.3s
	all	0.82	1		
Epoch 8/10	GPU_mem 0G	loss 0.2054	Instances 16	Size 128: 100%	25/25 10.2it/s 2.4s
	classes	top1_acc	top5_acc:	100%	4/4 17.5it/s 0.2s
	all	0.83	1		
Epoch 9/10	GPU_mem 0G	loss 0.2301	Instances 16	Size 128: 100%	25/25 9.0it/s 2.8s
	classes	top1_acc	top5_acc:	100%	4/4 16.9it/s 0.2s
	all	0.82	1		
Epoch 10/10	GPU_mem 0G	loss 0.1547	Instances 16	Size 128: 100%	25/25 9.6it/s 2.6s
	classes	top1_acc	top5_acc:	100%	4/4 15.8it/s 0.3s
	all	0.83	1		

10 epochs completed in 0.013 hours.

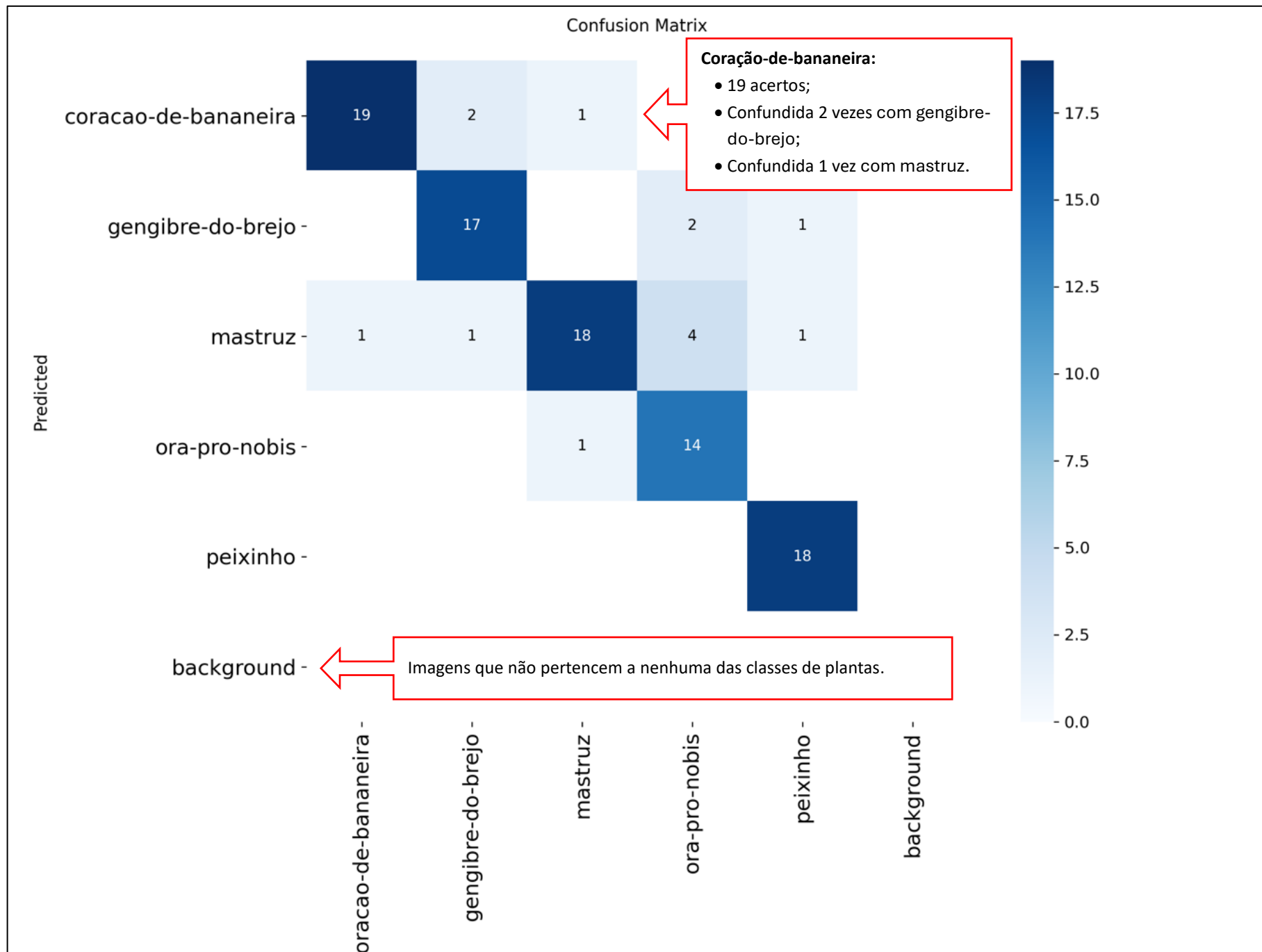
```

Epoch      GPU_mem      loss  Instances      Size
 9/10         0G      0.2301         16      128: 100% ————— 25/25 9.0it/s 2.8s
          classes  top1_acc  top5_acc: 100% ————— 4/4 16.9it/s 0.2s
          all      0.82           1

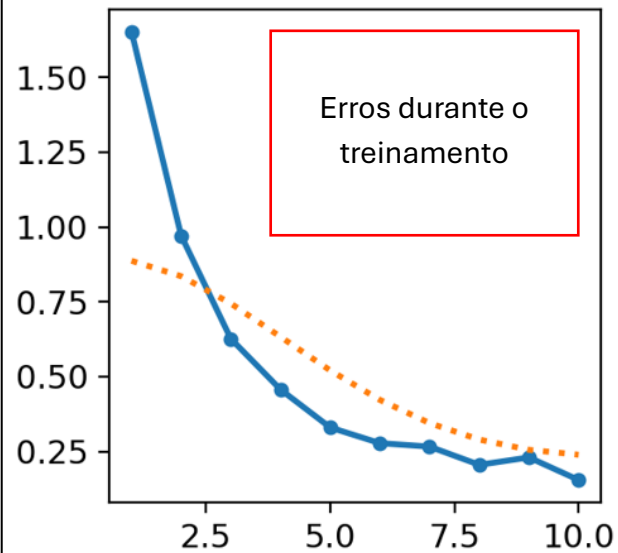
```

Métrica	Valor	Significado
Epoch	09/10	◆ Uma época → • Uma passagem completa do modelo • por todo o conjunto de imagens ◆ 9ª época de treinamento de um total de 10 épocas
GPU_mem	0G [Giga]	Nenhuma GPU em uso (não suporta <u>CUDA</u> [plataforma/modelo de programação que permite fazer <u>cálculos</u> gerais e <u>renderizar</u> (montar) gráficos] porque GPU não é da NVIDIA) [Intel(R) UHD Graphics]
classes	all	Métricas calculadas considerando todas as classes (• ora-pro-nóbis, • peixinho, • gengibre-do-brejo, • mastruz, • coração-de-bananeira)
loss	0.2301	◆ Diferença entre • o que o modelo <u>previu</u> e • o que <u>era o correto</u> ◆ É a média das perdas calculadas em cada amostra (imagem) do batch/lote ◆ Quanto menor o número, melhor
top1_acc	0.82	Accuracy Top-1 de 82% (a <u>principal previsão</u> está correta em 82% dos casos)
top5_acc	100%	Accuracy Top-5 de 100% (“A classe correta aparece entre as <u>5 previsões mais prováveis</u> ” - aqui, como existem 5 classes, o modelo faz 5 previsões (uma para cada classe))
Instances	16	Tamanho do batch/lote (<u>número de imagens processadas por vez</u> na validação)
Size	128	Tamanho das imagens de entrada (128 × 128 pixels [= 16384 pixels])
Validação	4/4 <u>16.9it/s</u> 0.2s	• 4 batches/lotes processados na <u>validação</u>; • <u>velocidade de 16,9 iterações/s</u>; • duração total de 0,2 s
Treinamento	25/25 <u>9.0it/s</u> 2.8s	• 25 batches/lotes processados no <u>treinamento</u>; • <u>velocidade de 9,0 iterações/s</u>; • duração total de 2,8 s

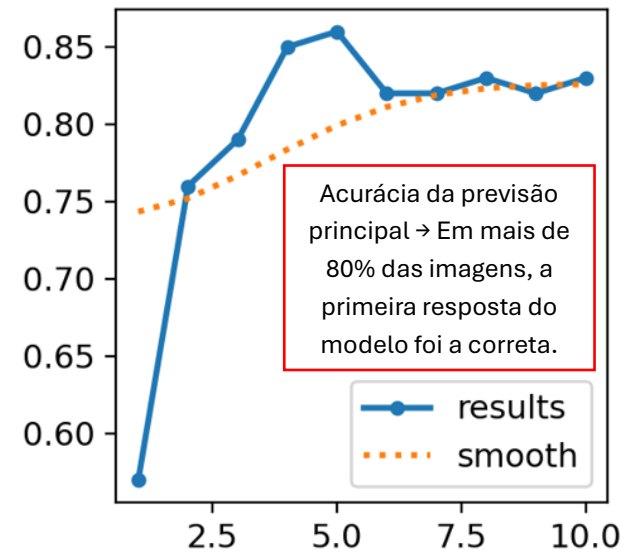




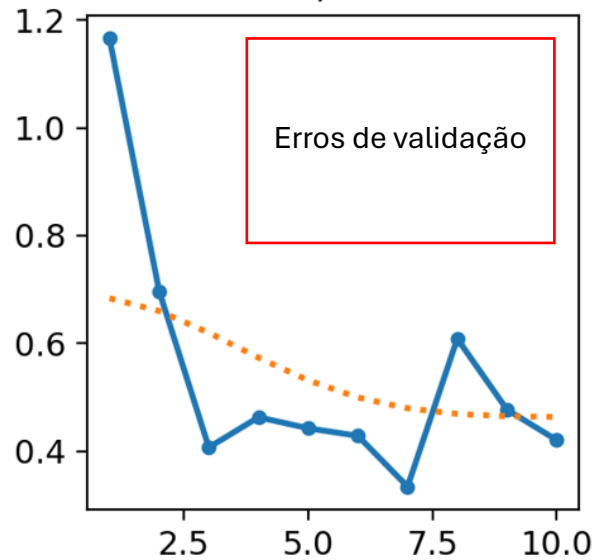
train/loss



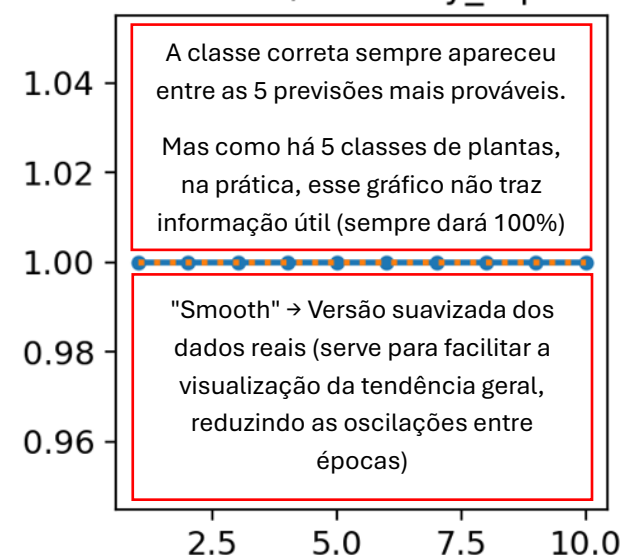
metrics/accuracy_top1

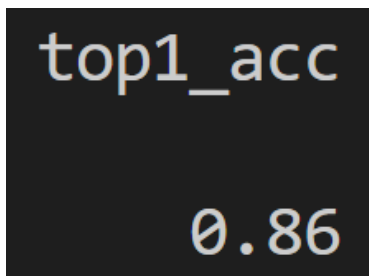


val/loss



metrics/accuracy_top5





86% ???

O que significa???

Significa que o modelo YOLO 11 n-cls:

- ✓ Utilizando •400 imagens para treinamento e •100 imagens para teste...
- ✓ Alcançou 86% de acurácia Top-1 (é a “principal previsão” – a de maior probabilidade de ocorrer)...
- ✓ Na classificação de 5 espécies de PANCs.

Ou seja:

- ✓ De cada 100 imagens...
- ✓ O modelo acertou cerca de 86 delas...
- ✓ Na primeira tentativa.

Supondo que a imagem seja:	
Ora-pro-nóbis	



Previsão do modelo	
Ora-pro-nóbis	72%
Mastruz	15%
Coração de bananeira	8%
Lírio do brejo	3%
Peixinho	2%

→ 1 acerto de 86 em 100

***Para um dataset de apenas 400 imagens de treino,
este é um resultado muito bom!!!***

Pois o modelo:

- ✓ Conseguiu aprender padrões consistentes...
- ✓ Com apenas 400 imagens...
- ✓ Que é uma quantidade considerada pequena...
- ✓ Para um problema de visão computacional.

Agradeço a atenção de todos!!!